

React Native & Expo

Interview Preparation Guide

A practical, level-wise breakdown of what Indian product companies, startups, and service firms ask in 2026.

Target roles	Junior / Mid / Senior React Native Developer
Frameworks	React Native (New Architecture), Expo SDK 55+
Audience	Candidates interviewing in India
Updated	May 2026

Interview content varies a lot by company tier and role level. This guide organizes the most common questions and topics into clear buckets so you can prepare strategically. Each section includes real interview topics, follow-up areas, and what interviewers typically probe deeper on.

Junior Level (0–2 years)

This round tests fundamentals: language basics, React core concepts, and the React Native primitives. Expect mostly conceptual questions plus small coding exercises.

JavaScript & TypeScript Basics

- **var vs let vs const**, hoisting, the temporal dead zone, closures, and how *this* is bound.
- **Promises**, `async/await`, the event loop, microtasks vs macrotasks.
- Array methods: `map`, `filter`, `reduce`, `find`, `some`, `every`. Spread / rest operators, destructuring.
- TypeScript: interface vs type, generics, optional chaining, utility types (`Partial`, `Pick`, `Omit`).

React Core

- Functional vs class components, and why hooks replaced classes.
- Difference between props and state. Why state updates are asynchronous.
- `useState`, `useEffect`, `useRef`, `useMemo`, `useCallback` — what each does and when to use it.
- Why **keys** are needed in lists, and what happens if you use an array index as key.
- Controlled vs uncontrolled components, and lifting state up.

React Native Specifics

- Difference between React and React Native (DOM vs native bridge to `UIView/ViewGroup`).
- Core components: `View`, `Text`, `Image`, `ScrollView`, `FlatList`, `TextInput`, `Pressable`, `TouchableOpacity`.

- **FlatList vs ScrollView** — when to use which. (FlatList virtualizes; ScrollView renders all at once.)
- Flexbox in RN: default flexDirection is **column**, not row like on the web — a classic gotcha.
- StyleSheet.create vs inline styles — reference stability and bridge optimization.
- SafeAreaView and why it matters for notches and the iPhone home indicator.
- Platform.OS, Dimensions, PixelRatio for responsive layouts.

Expo Basics

- What Expo is, and the difference between the managed and bare workflows.
- Expo Go vs Development Build — why dev builds matter for custom native modules.
- EAS Build, EAS Submit, EAS Update at a high level.

Tip: Junior interviewers love asking you to *build a small UI live* — a card, a counter, a form field. Practice typing this from scratch without copy-paste.

Mid Level (2–4 years)

Where most hiring happens in India. Questions go deeper into navigation, state management, performance, and the New Architecture.

Navigation

- React Navigation: Stack, Tab, Drawer navigators and when to combine them.
- Passing params between screens, deep linking, and universal links.
- Nested navigators, useNavigation, useRoute, navigation prop typing in TypeScript.
- **Expo Router** (file-based routing) — increasingly asked in 2026.
- Auth flows: how to protect routes and handle redirects after login.

State Management

- Context API vs Redux Toolkit vs Zustand vs Jotai — when to use each.
- Redux Toolkit: slices, createAsyncThunk, RTK Query.
- **React Query / TanStack Query** — the server-state vs client-state distinction.
- AsyncStorage vs MMKV vs expo-secure-store for different persistence needs.

Performance

- Why FlatList is faster than mapping inside a ScrollView (virtualization).
- getItemLayout, keyExtractor, removeClippedSubviews, initialNumToRender, windowSize.
- React.memo, useMemo, useCallback — and the new React Compiler caveat in React 19.
- Avoiding unnecessary re-renders with stable references and split contexts.
- **FlashList** (Shopify) vs FlatList for very large lists.
- Image optimization with expo-image instead of the built-in Image component.
- **Hermes engine** — what it is and why it ships by default now.

Networking & Data

- fetch vs axios, interceptors, retries, request cancellation.
- Handling token refresh cleanly without race conditions.
- Offline-first apps, cache strategies, optimistic updates.
- WebSockets and real-time updates (Socket.IO, native WebSocket).

The New Architecture (Hot Topic in 2026)

Almost every mid-to-senior interview now asks about this. Be ready to explain:

- **The old bridge:** JSON-serialized async messages between JS and native threads, the source of many performance issues.
- **JSI (JavaScript Interface):** direct synchronous JS-to-native calls, no JSON serialization.
- **Fabric:** the new rendering system — synchronous layout and concurrent rendering.
- **TurboModules:** lazy-loaded native modules with type-safe interfaces.

- **Codegen:** generates native interface code from JS specs.
- Threading model: JS thread, UI thread, Shadow thread — and what runs where.

Animations

- The Animated API basics and its limitations.
- **react-native-reanimated:** worklets, `useSharedValue`, `useAnimatedStyle`, `withSpring`, `withTiming`.
- react-native-gesture-handler for complex gestures (Pan, Pinch, Long Press).
- Why Reanimated runs animations on the UI thread, bypassing the JS thread.

Expo & Deployment

- OTA (over-the-air) updates with expo-updates and EAS Update.
- Push notifications: Expo Notifications, APNs (iOS), FCM (Android).
- App signing, building for stores, internal testing tracks.
- Config plugins and the prebuild workflow.

Senior Level (4+ years)

Architecture, trade-offs, and ownership dominate. Expect open-ended questions and system-design rounds.

- Designing a scalable folder structure for a large RN app (feature-based vs layer-based).
- Monorepo setup (Turborepo, Nx, pnpm workspaces) for sharing code between web and mobile.
- Code-push strategies, gradual rollouts, and rollback handling.
- Writing a custom native module when no library exists — Java/Kotlin (Android) or Swift/Obj-C (iOS).
- Migrating an existing app to the New Architecture step by step.
- Finding memory leaks: Flipper, Xcode Instruments, Android Studio profiler.
- CI/CD pipelines with EAS Build, Fastlane, or GitHub Actions.
- Security: certificate pinning, code obfuscation, jailbreak/root detection, token storage.
- Accessibility: screen readers, focus order, color contrast, font scaling.
- Internationalization (i18n) with i18next or expo-localization.
- A/B testing setup with Firebase, Statsig, or LaunchDarkly.

Tip: Senior interviews often turn into *conversations*. Have strong opinions backed by reasons. "I prefer Zustand over Redux Toolkit for small apps because..." is far more impressive than reciting definitions.

Coding Round (DSA + Practical)

Most Indian product companies include a coding round. There are usually two flavors:

DSA Round

Same as any frontend role: arrays, strings, hash maps, two pointers, sliding window, trees, basic DP. Usually 1–2 medium LeetCode-style problems in 45 minutes. Companies like Swiggy, Zomato, Cred, PhonePe, and Razorpay lean heavily on this.

Machine Coding Round

Build a small React Native app in 60–90 minutes. Common prompts include:

- A todo list with add / delete / edit and AsyncStorage persistence.
- A search bar with debounce that hits a real API.
- An infinite-scroll feed with pull-to-refresh.
- A timer or stopwatch with start / pause / reset.
- A movie or recipe browser app using a public API like TMDB or TheMealDB.
- A form with validation using React Hook Form + Zod (or Formik + Yup).

Interviewers watch your folder structure, component splitting, hook usage, error handling, loading states, and (sometimes) Git commit hygiene.

Company-Specific Patterns

Company Type	Examples	What They Focus On
Service companies	TCS, Infosys, Wipro, Accenture, Cognizant, HCL	Heavy on theory and definitions. Less DSA. SDLC, Agile, project e
Product startups	Cred, PhonePe, Swiggy, Zomato, Meesho, Chirp, Zing, Grow, Droom	DSA, algorithms, coding, growth, product design (senior). Performance, anim
Agencies / app studios	Smaller firms doing client work	Expo expertise, EAS, OTA updates, payment integrations, push not
FAANG-adjacent	Flipkart, Myntra, Walmart Global Tech, Amazon	Strong DSA, system design, deep React internals, occasional native

Behavioral & Project Discussion

This round almost always happens, even at startups. Be ready to:

- Walk through a project end-to-end, including **why you chose RN** over Flutter or native.
- Describe a tough bug you fixed — memory leak, crash, performance regression.
- Explain trade-offs you made (Expo Go vs dev build, Redux vs Zustand, REST vs GraphQL).
- Talk about deployment experience: Play Store, App Store, internal testing, beta tracks.
- Describe a time you disagreed with a teammate or PM about a technical decision.

Quick-Fire Questions to Practice

These come up again and again. Try writing your own one-paragraph answer for each, then refine it.

- Explain the React Native New Architecture in 60 seconds.
- What is the difference between FlatList and ScrollView?
- Why is StyleSheet.create preferred over inline styles?
- What is the difference between Expo managed and bare workflow?
- When would you choose Zustand over Redux Toolkit?
- How does react-query decide when to refetch data?
- What is the role of Hermes in React Native?
- How do you debug a performance issue in a list-heavy screen?
- What is the difference between Pressable and TouchableOpacity?
- How do you persist data securely on the device?
- How do you handle deep linking in Expo Router?
- What is the difference between useMemo and useCallback?
- How do worklets work in react-native-reanimated?
- What happens when you call setState — explain the render cycle.
- How do you handle keyboard avoiding behavior in forms?

Practical Prep Checklist

	Action
1	Build 2–3 polished projects on GitHub. Interviewers love walking through real code.
2	Know the New Architecture conceptually — JSI, Fabric, TurboModules.
3	Be honest about Expo. Pick managed or bare, know one deeply, understand the other.
4	Practice 60-minute machine-coding builds. Common at product companies.
5	Read the official React Native and Expo docs once cover-to-cover.
6	Practice DSA on LeetCode — focus on arrays, strings, hash maps, trees.
7	Prepare 2–3 strong project stories using the STAR format.
8	Set up a personal app on the Play Store or App Store — shows real shipping experience.
9	Learn one animation library deeply (Reanimated is the right pick in 2026).
10	Mock interview with a peer at least 3 times before the real thing.

Final Note

Interviews are as much about confidence and clarity as raw knowledge. Speak slowly, think out loud, and don't be afraid to say "I don't know, but here's how I'd find out." Indian product companies in particular value problem-solving approach over memorized answers.

Good luck — you've got this.